PHP for Beginners

Ep 3: "Your first PHP Tag"

i) Creating directory for the website. Depending on environment setup, designated folder will vary [1:05]

ii) Created index.html then switch to index.php [0:26]

iii) PHP syntax begins with using the <? php ... ?> [0:36]

Ep 4: Variables:

i) Importance of variables - can be manipulated or 'pointed to' [4:39]

ii) opening and closing PHP Tags - can open with ~~<?~~ <code><?PHP</code> but end with different - everything inside will be interpreted as ~~PHP~~ PHP. [0:33]

iii) String concatenation: ~~use~~ use <code>.</code> to concatenate strings.

```
$greeting = "Hello";
echo $greeting . "everybody";
```

* Why is are variables important?

Ep.5: Conditionals and Booleans

i) concept of conditionals as a way to create branches in logic [04:12]

ii) classic use of boolean values for `true` or `false` [04:28]

iii) Dynamic redering messages based on conditional checks [05:00]

```
if ($ read) {
    $ message = "You have", $ name;
} else {
    $ message = "You have not", $ name;
}
```

* how do conditionals help in making decisions?

EP 6: Arrays

i) Arry is like a folder that holds a collection of items [02:03]

ii) Using [] notation we can create array and and store multiple values as string[]. [02:29]

iii) Using foreach we can iterate through a items in an array - effectively loop through the array [02:32]

```
$ books = [ "The langoliers", "Hall Mary", ... ];
foreach ($books as $book) {
    echo "<li>{$book}</li>";
}
```

Ep.7: Associative Arrays

Ans i) Associative arrays allow us to associate keys with values, making data key-value pairs [03:39]

ii) Accessing items in an associative array is done using the key instead of an index [02:53]

iii) When looping through an associative array, each item is an array itself, so we must specify the key to get the desired value.

```
$books = [
    ['name' => 'The Lanyllers', 'author' => 'Stephen'],
    ['name' => 'Project Hail Mary', 'author' => 'Andy Weir'],
];
// looping associative array
foreach ($books as $book){
    echo $book['name']; // assessing the name using
}                       the key,
```

EP 16 Create a MySQL Database

i) CREATE DATABASE myapp; - to create database [0:35]

ii) Defining a table - [7:15]

EP 17 PDO First Steps

i) PDO - PHP Data objects - lightweight interface for accessing databases [0:20]

ii) connecting to MySQL database requires a new instance of the PDO class which in turn requires a DSN - data source name string an other parameters [8:45]

iii) fetchAll() is used to retrieve data effectively after executing a query [11:48]

```
$dsn = 'mysql:host=localhost;port=3306;dbname=myAPP;charset=utf8';
$username = 'root';
$password = '';

try {
    $pdo = new PDO($dsn, $username, $password);
    $statement = $pdo->prepare('SELECT * FROM posts');
}
```

REP-20 SQL Injection vulnerability Explained

i) Always assume user input is malicious! [00:39]

ii) A user could easily manipulate the query to execute damaging commands [DROP TABLE users;]

iii) To secure our application, we use prepared statements instead of inlining user data in queries.

```
$query = "SELECT * FROM posts WHERE id=?";
```

EP 25 Intro to Forms and Request Methods

~~i) By default~~

i) Default forms submit request GET is not ideal -> we use POST request ~~for~~ to avoid unintented changes in records. [11:06]

ii) $_POST super global allows access to submitted form attributes. [13:21]

[Superglobal variable in PHP are built-in, predefined arrays that are always accessible in all scopes without needing special global declarations]

iii) Each input needs a name attribute ~~for~~ to form ~~data~~ to capture and submit data. Without the attribute the data won't be included in the submission [~~11:00~~] [10:52]

```
<form method = "POST">
  <label for = "body">Note:</label>
  <textarea id = "body" ~~no~~ name = "body"></textarea>
  <button type = "submit">Create Note</button>
</form>
```

EP 26 : Always Escape Untruste Input

i) use Prepared statements to prevent SQL Injection [04:42]

[Prepared statement = a feature in DBMs that Separates the SQL code structure from the actual user data]

ii) use htmlspecialchars to sanitize user input before displaying on the page [06:48] (& -> &) (" -> ")

iii) validation to ensure user input meets expectations [08:15]

~~$ sat~~ $sanitized Input = htmlspecialchars($userInput, ENT_QUOTES, 'UTF-8');

EP 27: Intro to Form Validation

i) validation is necessary to prevent unwanted entries such as submitting empty data.

ii) ~~Server~~ Server side ~~validat~~ validation is also implemented in case client side validation is bypassed [3:00]

iii) Error reporting using errors array to store any validation messages - feedback to user. [5:10]

$errors['body'] = "a body is required";